Custom Hooks: Building Reusable Logic

6 min read Updated 24 Mar 2026 Topic 14 of 62

2.4 – Custom Hooks: Building Reusable Logic

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"What is a custom hook?"	🟡	★★★★★
"Build a useFetch / useDebounce / useLocalStorage hook"	🟡🔴	★★★★★
"What are the rules of hooks?"	🟢🟡	★★★★☆
"Why must custom hooks start with 'use'?"	🟡	★★★☆☆
"How is a custom hook different from a regular function?"	🟡	★★★☆☆

What Is a Custom Hook?

A custom hook is a JavaScript function whose name starts with `use` and that calls other hooks inside it. That's the entire definition. There's no special API, no registration, no magic. If a function starts with `use` and calls `useState`, `useEffect`, or any other hook, it's a custom hook.

The point is **reusability**. When you find the same useState + useEffect combination repeated across components, extract it into a custom hook.

Rules of Hooks

These apply to all hooks, including custom ones:

1. **Only call hooks at the top level.** Not inside conditions, loops, or nested functions. React relies on call order to track which hook is which.
2. **Only call hooks from React functions.** Either from functional components or from other custom hooks. Not from regular JavaScript functions.
3. **Custom hooks must start with `use`.** This lets React's linter enforce the rules above.

JSX Copy

```jsx
// ❌ Hook inside a condition. Breaks hook ordering.
function Bad({ shouldFetch }) {
  if (shouldFetch) {
    const [data, setData] = useState(null);  // React can't track this
  }
}

// ✅ Always call hooks at the top. Use the condition inside.
function Good({ shouldFetch }) {
  const [data, setData] = useState(null);

  useEffect(() => {
    if (shouldFetch) {
      fetchData().then(setData);
    }
```

Five Custom Hooks Every React Developer Should Know

1. useLocalStorage

Syncs state with localStorage so it persists across page refreshes.

JSX Copy

```jsx
function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    try {
      const stored = localStorage.getItem(key);
      return stored !== null ? JSON.parse(stored) : initialValue;
    } catch {
      return initialValue;
    }
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

// Usage
function Settings() {
  const [theme, setTheme] = useLocalStorage('theme', 'light');
  return <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'lig
}
```

2. useDebounce

Delays updating a value until the user stops changing it. Essential for search inputs.

```jsx
function useDebounce(value, delay = 500) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => setDebouncedValue(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);

  return debouncedValue;
}

// Usage
function Search() {
  const [query, setQuery] = useState('');
  const debouncedQuery = useDebounce(query, 300);

  useEffect(() => {
    if (debouncedQuery) {
      searchAPI(debouncedQuery);
    }
  }, [debouncedQuery]);

  return <input value={query} onChange={e => setQuery(e.target.value)} />;
}
```

3. useFetch

Handles the loading/data/error lifecycle for API calls.

JSX Copy

```jsx
function useFetch(url) {
  const [state, setState] = useState({ data: null, loading: true, error: n

  useEffect(() => {
    if (!url) return;
    let cancelled = false;

    setState({ data: null, loading: true, error: null });
```

```
      return res.json();
    })
    .then(data => {
      if (!cancelled) setState({ data, loading: false, error: null });
    })
    .catch(error => {
      if (!cancelled) setState({ data: null, loading: false, error: erro
    });

  return () => { cancelled = true; };
}, [url]);

return state;
}

// Usage
function UserList() {
  const { data: users, loading, error } = useFetch('/api/users');

  if (loading) return <p>Loading ...</p>;
  if (error) return <p>Error: {error}</p>;
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

4. useWindowSize

Tracks the browser window dimensions in real time.

JSX Copy

```
function useWindowSize() {
  const [size, setSize] = useState({ width: window.innerWidth, height: win

  useEffect(() => {
    const handleResize = () => setSize({ width: window.innerWidth, height:
    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);
```

```
// Usage
function ResponsiveNav() {
  const { width } = useWindowSize();
  return width > 768 ? <DesktopNav /> : <MobileNav />;
}
```

5. useClickOutside

Detects clicks outside a referenced element. Used for closing dropdowns, modals, and popovers.

JSX Copy

```
function useClickOutside(ref, callback) {
  useEffect(() => {
    const handleClick = (e) => {
      if (ref.current && !ref.current.contains(e.target)) {
        callback();
      }
    };

    document.addEventListener('mousedown', handleClick);
    return () => document.removeEventListener('mousedown', handleClick);
  }, [ref, callback]);
}

// Usage
function Dropdown() {
  const [open, setOpen] = useState(false);
  const ref = useRef(null);

  useClickOutside(ref, () => setOpen(false));

  return (
    <div ref={ref}>
      <button onClick={() => setOpen(!open)}>Menu</button>
      {open && <div className="dropdown-content">Options here</div>}
    </div>
```

How a Custom Hook Differs from a Regular Function

A regular utility function can do math, string manipulation, or data transformation. But it **cannot** call hooks. A custom hook can call `useState`, `useEffect`, `useRef`, and any other hook because React recognizes it as part of a component's lifecycle (via the `use` prefix).

JSX Copy

```jsx
// Regular function: no hooks allowed
function formatPrice(amount) {
  return `$${amount.toFixed(2)}`;
}

// Custom hook: can use hooks
function useFormattedPrice(amount) {
  const [formatted, setFormatted] = useState('');
  useEffect(() => {
    setFormatted(`$${amount.toFixed(2)}`);
  }, [amount]);
  return formatted;
}
// Note: this example is over-engineered. formatPrice as a regular functio
// Only use a custom hook when you actually need hooks inside.
```

⚡ Common Mistakes

Mistake 1: Making a custom hook for something that doesn't need hooks. If your function doesn't call useState, useEffect, or any other hook, it's just a utility function. Name it without the `use` prefix.

Mistake 2: Not handling cleanup. If your hook sets up listeners, timers, or subscriptions, it must clean them up in the useEffect return function.

Mistake 3: Breaking hook rules inside the custom hook. The same rules apply: no hooks inside conditions or loops, even within a custom hook.

Interview Answer

> "A custom hook is a function that starts with 'use' and calls other React hooks inside it. It lets you extract and reuse stateful logic across components without changing the component hierarchy. The most common ones I build are useFetch for API calls, useDebounce for search inputs, useLocalStorage for persisting state, and useClickOutside for dropdown menus. The key difference from a regular function is that a custom hook can call useState, useEffect, and other hooks, which a plain function cannot."

Key Takeaways

- A custom hook is a function starting with `use` that calls other hooks.
- It extracts **reusable stateful logic**, not reusable UI (that's what components do).
- The same rules apply: top-level only, React functions only.
- Build hooks for repeated patterns: useFetch, useDebounce, useLocalStorage, useWindowSize, useClickOutside.
- Don't create a custom hook if you don't need hooks inside. Just use a regular function.
- Always handle **cleanup** for subscriptions, timers, and event listeners.

QUICK CHECK — no marks, just for you.

1. ... useSlugify(title) that does string manipulation and returns a slug. It calls no other hooks. According to the lesson, what is it?

- A valid custom hook, because any function called from a component is a hook
- An invalid custom hook that React will throw an error on at runtime
- A valid custom hook, because the use prefix is what registers it with React
- A regular utility function that should be renamed without the use prefix

Check answer

2. The lesson shows a component that calls useState only inside an if (shouldFetch) block. What is the reason the lesson gives for why this breaks?

- React only allows one useState call per component, and the condition creates a second one
- React tracks hooks by the variable name they are assigned to, and the name is scoped to the if block
- State declared inside a block is garbage collected when the block exits
- React tracks hooks by their call order, so a conditional call makes the ordering unreliable

Check answer

3. In the lesson's useDebounce hook, the useEffect returns a function that calls clearTimeout on the pending timer. Per the lesson, what makes this return function necessary?

- clearTimeout is what actually schedules the debounced update after the delay
- A hook that sets up timers, listeners, or subscriptions must clean them up in the useEffect return function

Without it the dependency array would be ignored and the effect would run on every render

Check answer

Previous
useMemo & useCallback: Performance Hooks

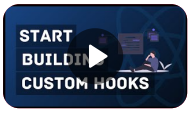
Next
React.memo, PureComponent & Re-render Prevention

LEARN MORE

ARTICLES & DOCS

Reusing Logic with Custom Hooks
react.dev

VIDEOS



Custom React hook in 12 minutes
YouTube



Making a custom hook
YouTube



Custom hooks — deep dive
YouTube

Hand-picked references for this lesson.